畢業專題 文獻探討

Agentic AI Security

Syu

October 5, 2026

Index

1. 回顧
2. Agentic AI Security (arXiv 2510.23883)
3. Agentic AI and Cybersecurity (arXiv 2601.05293)
4. Agent-SafetyBench (arXiv 2412.14470)

回顧

1. A Defensive Framework Against Adversarial Attacks on Machine Learning-Based Network Intrusion Detection Systems (arXiv 2502.15561)
2. Adversarial Challenges in Network Intrusion Detection Systems: Research Insights and Future Prospects (arXiv 2409.18736)

Agentic AI Security (arXiv 2510.23883)

Agentic AI Security: Threats, Defenses, Evaluation, and Open Challenges
Chhabra, Datta, Nahin, Mohapatra (University of South Florida)

摘要

Agentic AI 具備規劃、工具使用、記憶與自主性，能在網頁、軟體與實體環境中自動執行任務， 也因此產生不同於傳統 AI 安全與軟體安全的新風險。本文為 survey：整理威脅分類、防禦策略、評測方法與開放問題，目標是支援 secure-by-design 的 agent 系統。

論文結構

威脅分類 (§3) → 防禦與安全控制 (§4) → Benchmark 與評估 (§5) → 開放問題 (§6)

動機：為什麼 Agentic AI 的安全不同

- 一般 LLM 多為被動回應；Agent 具備持久記憶、跨時間推理、與其他 agent 協作
- 自主性與持久性 → 攻擊面擴大；工具整合 → 放大濫用；多 agent 協作 → 行為難預測

真實案例

- **EchoLeak (CVE-2025-32711)**：惡意郵件中的 prompt 讓 Microsoft Copilot 在無使用者互動下外洩資料
- Symantec 以 OpenAI Operator 實驗：可蒐集個資並自動化 credential stuffing

Lupinacci et al. 的測試結果

94.4% 的 SOTA LLM agent 易受 prompt injection，83.3% 易受 retrieval-based backdoor，100% 易受 inter-agent trust exploit

威脅分類總覽

類別	主要內容
Prompt Injection & Jailbreaks	直接/間接、有意/無意、依模態、依傳播性、多語言與混淆、payload splitting
Autonomous Cyber-Exploitation & Tool Abuse	one-day 漏洞利用、自主網站入侵、emergent tool abuse
Multi-agent & Protocol-level	MCP / A2A 協定攻擊、冒充、協作操弄、責任歸屬模糊
Interface & Environment Risks	觀察/行動空間錯位、perception-action　脆弱性、動態內容與 CAPTCHA
Governance & Autonomy	人類監督不足、自主等級、治理框架

各類別並非互斥，依主要特性歸類以利分析。

Prompt Injection：直接與間接

Direct (DPI)

- 惡意指令直接放入 prompt
- 攻擊者 = 終端使用者
- 例：「ignore all previous instructions…」，誘使客服 bot 洩漏內部資料或寄信

Indirect (IPI)

- 惡意指令藏在網頁、API、資料庫等外部資料
- 攻擊者 = 第三方資料提供者
- 更隱蔽：看起來像合法指令
- Adaptive attack 對 8 種 IPI 防禦的成功率約 50%

其他分類維度	說明
有意 / 無意	含糊的使用者輸入或長對話的 context drift 也可能覆蓋系統指令
模態	文字（code / P2SQL）、圖像/影片、音訊、混合（CrossInject 效果至少提升 30.1%）
傳播性	非傳播 vs. 傳播（recursive injection、AI worm）
混淆與拆分	Base64、emoji、低資源語言；payload splitting 將惡意內容拆成多段，聚合時才成形

自主網路利用與工具濫用

- **One-day 漏洞**：GPT-4 agent 在提供 CVE 描述時成功率 87%，優於 OWASP ZAP、Metasploit 等傳統工具
- **自主網站入侵**：在不預知漏洞下完成 XSS+CSRF 串接、SSTI、blind SQL union injection；缺少文件存取或詳細 prompt 時效能大降
- **Emergent tool abuse**：ConAgents（多 agent 分工選工具、執行、校正）、ToolGen（把工具呼叫放進 next-token 生成）使工具串接更容易

攻擊經濟學

每次 one-day exploit 成本僅數美元，且可平行化，大規模攻擊比雇用人類攻擊者便宜

⇒ 需要 containment 與 runtime monitoring

多代理與協定層威脅

MCP（client–server）

- Flooding / replay → DoS
- Token 外洩或不安全 proxy → 冒充 agent
- Embedded backdoor、retrieval corruption
- 推論與 side-channel 導致機密外洩

A2A（agent–agent）

- 假 agent 宣告、未授權註冊
- 遞迴任務委派 → deadlock / 無限迴圈 DoS
- Transitive prompt injection
- Context tampering、memory manipulation

Threat actor 視角的六類威脅（改編自 Ko et al.）

Impersonation & Role Abuse、Coordination Manipulation、Knowledge & Learning Manipulation、Inference & Policy Evasion、Accountability Obfuscation、Confidentiality & Integrity Breaches

介面環境風險與治理

- **觀察/行動空間錯位**：LLM 以靜態文字預訓練，卻要做 scroll、hover、切分頁；AgentOccam 簡化動作空間後穩定度提升
- **Perception-action 脆弱**（WebArena）：
  - 誤解先前輸入、重複相同搜尋直到步數用盡
  - 提前終止：GPT-4 將 54.9% 可完成的任務誤判為不可能
  - 模板脆弱：僅在 61 個模板中的 4 個穩定成功
- **動態內容與 CAPTCHA**：多模態 agent 最高約 40% vs. 人類近 100%
- **治理與自主性**：低監督下可能出現不可預測行為與劫持；建議 human-in-the-loop 與結構化自主等級

防禦 (1)：Prompt Injection 抵抗設計

類型	代表方法	取捨
Agent-focused	Instruction hierarchy、StruQ、SecAlign (DPO)、circuit breaking	需重訓，可能降低通用能力，對 adaptive attack 效果有限
System-focused	Dual-LLM、Plan-then-Execute、Melon、TrustAgent；偵測、隔離、prompt augmentation、perplexity	保留模型通用性但系統更複雜，多階段/多 agent 攻擊仍可繞過
User-focused	人類驗證	無法擴展，攻擊者就是使用者時失效

結論

完全阻止 prompt injection 仍是開放問題；實務上傾向混合式 (hybrid) 設計，組合不同類型的防禦。

防禦 (2)：政策執行、沙箱、監控、標準

- **Policy enforcement**
  - Governance-centric（runtime）：GuardAgent、AgentSpec（DSL 定義規則）、ShieldAgent、R^2-Guard
  - Signal-centric（掃描輸入輸出）：Llama Guard、LlavaGuard、Safewatch
- **Sandboxing**：SandboxEval、容器化（gVisor / Docker）、ToolEmu；限制：沙箱本身有漏洞（跨 session 洩漏）、降低效率
- **Detection & monitoring**：Pro2Guard 以機率可達性分析預測違規；Adversarial Markov Games 讓攻防共同演化
- **標準**：NIST AI RMF GenAI Profile、OWASP Agentic AI Threats、CSA MAESTRO

Table 1 的觀察

21 個方法中，涵蓋 Rate limiting 與 Rollback/Recovery 的各只有 3 個，多數集中在輸入治理與政策層。

Benchmark 與評估

Benchmark 類型

- 能力導向：BrowserGym、τ-bench、OSWorld、AndroidWorld
- 安全導向：ST-WebAgentBench、AgentHarm、OS-Harm、R-Judge、ToolEmu、AgentDojo、InjecAgent、ASB、Agent-SafetyBench

評估方式的演進

- Process-aware：評分軌跡而非只看最終結果
- 重複試驗：pass$^{\wedge k}$（GPT-4o 在 retail 的 pass$^{\wedge 8}$ 可低於 25%）
- 標準化 LLM judge：回報與人類一致性
- 沙箱/模擬器需說明擬真度
- 可重現性：釋出攻擊模板、回報成本與延遲

開放問題與結論

1. **長時程安全**：時間穩健性、潛伏式不當行為偵測
2. **多 agent 安全**：單一惡意 agent 可造成連鎖失敗；需認證通訊、回滾機制、sentinel agent
3. **更好的 benchmark**：分布式指標、可靠的 LLM judge、高擬真沙箱
4. **Adaptive attack**：多數防禦只做靜態評估
5. **實體世界**：感測器欺騙（GPS、LiDAR）、端到端 VLA 模型缺乏防禦分層
6. **人機介面**：人類監督本身可被社交工程影響

Agentic AI and Cybersecurity (arXiv 2601.05293)

A Survey of Agentic AI and Cybersecurity: Challenges, Opportunities and Use-case Prototypes

Lazer, Aryal, Gupta, Bertino（Tennessee Tech / Univ. of Nebraska Omaha / Purdue）

問題

Agentic AI 在資安上是雙面刃：能強化持續監控、事件回應、威脅獵捕；同樣的能力也加速偵察、利用與社交工程。 現有治理與保證機制多假設短生命週期、有人在迴圈的 AI，無法涵蓋 collusion、cascading failure、memory poisoning、oversight evasion 等系統性風險。

貢獻

概念基礎 → 防禦應用 → 攻擊應用 → 系統性風險 → 框架與 benchmark → 量子考量 → 三個原型實作

什麼是 Agentic AI：架構與自主等級

核心特性

- Reasoning（CoT、Reflection）
- Interaction（工具、API、程式執行）
- Autonomy
- Adaptability（記憶與回饋）

以 foundation model 為核心，加上記憶、檢索 (RAG)、工具，形成規劃–行動–反思的迴圈；多 agent 架構由 coordinator 分派給 task agents。

Level	自主程度	資安意涵
0	Static inference	風險最低、可預測
1	Assistive	低風險
2	Tool-assisted (RAG)	中等風險
3	Semi-agentic	高風險
4	Fully agentic	極高、難稽核

防禦應用

領域	內容
Autonomous cyber defense	自主監控與偵測、事件回應、協同防禦
Threat intelligence & adversarial analysis	主動威脅獵捕、攻擊模擬（滲透測試、CTF）、對抗式 AI 防禦（釣魚等）
Enterprise automation & governance	DevSecOps、IAM、資安人力擴增
Simulation, training & testing	自動生成 cyber range（ARCeR）、高擬真訓練環境（Cyberwheel）

- 多為模組化 multi-agent，各 agent 對應 NIST 單一功能（Govern / Identify / Protect / Detect / Respond / Recover）
- 取捨：速度 vs. 執行權限；不可逆動作需人類授權（Knack & Burke）
- 定位為擴增而非取代分析師（全球資安人力缺口約 4–5 百萬）

攻擊應用：Agentic AI 帶來的攻擊

攻擊領域	重點
Ransomware	Unit 42 框架模擬完整攻擊鏈約 25 分鐘；外洩平均時間由 2021 年 9 天降至 2024 年約 2 天
Insider threat	被入侵的 agent 以合法憑證長期持續執行看似正常的動作
Social engineering & fraud	ScamAgents 可進行多輪詐騙電話並規避 guardrail
Exploitation & recon	HexStrike-AI 被濫用於平行掃描數千個 IP

論文觀點

Agentic ransomware 是自動化與決策自主的升級，而非新的密碼學或漏洞類別；調整發生在 orchestration 層。重點轉向「在傳播與外洩完成前打斷自主攻擊迴圈」。

Agentic AI 本身的安全：四層風險模型

層	威脅	防禦
Perception	Data poisoning、adversarial input、supply chain	Input validation、guardrails、anomaly detection
Reasoning	Prompt injection、logic manipulation、backdoor	Multi-agent debate、reasoning guardrails
Action	API 濫用、unsafe tool invocation、未授權程式執行	Sandboxing、privilege separation
Memory	Poisoned retrieval、資料外洩	加密、細粒度存取控制

來源：Wong & Saade。失敗會跨層傳播，因此需要跨層協同的控制。 其他框架：ATFAA / SHIELD、MAESTRO、OWASP Agentic Security Initiative、Microsoft 失敗模式分類。

系統層漏洞與 Prompt Injection 緩解

He et al.：三類系統層漏洞

Session 管理失敗、model pollution 與隱私外洩、可執行 agent 程式導致任意程式碼執行。 Bash agent 實驗：未使用沙箱時超過 75% 惡意指令可成功執行，容器沙箱則幾乎全部擋下。

緩解類型	機制	限制
Input sanitization	擋明顯樣式的注入	對語意、間接、多模態注入無效
Least-privilege 憑證	縮小爆炸半徑	無法阻止在允許範圍內被誤導
Runtime monitoring / red teaming	偵測不安全行為	多為事後，難抓跨工具連鎖失敗
Intent-bound delegation	行動綁定授權意圖	依賴上游意圖正確指定

多代理、資源濫用、推理與記憶

多代理安全

- 共享記憶/權限讓單一被入侵 agent 引發連鎖；隱寫術可做隱蔽協作
- 推理層防禦：PeerGuard、debate / voting
- 基礎設施層：GUARDIAN、SentinelAgent（圖形異常偵測）
- BlockA2A、SAFEFLOW 做身分與資訊流控制
- Multi-agent security tax：控制愈強，協作效率愈低

其他風險

- **資源濫用**：CRABS 類攻擊以合法輸入引發過量 token 與工具呼叫，成本放大式 DoS
- **推理劫持**：竄改 reasoning trace 重導多步規劃
- **記憶污染**：污染可在攻擊結束後仍影響未來任務
- 人機層面：agent 代為點擊、下載、輸入憑證，擴大社交工程面

原型實作（Mistral via Ollama，合成資料）

原型	設定	結果
Red / Blue agents	讀系統設定檔，兩輪攻防	雙方依對手上一輪行為自行調整策略
SOC triage agent	6 筆合成警報、policy gate 限制唯讀	Baseline：P 1.00, R 0.75, F1 0.86, MTTR 7.5 分；Agent：P / R / F1 皆 1.00, MTTR 6.0 分
Anomaly / IDS agent	合成 NetFlow + RAG notes	Baseline（z-score + IForest）：AUROC 0.87, F1 0.74；Agent：AUROC 0.93, F1 0.81

個人註記：實驗為可行性展示，資料為合成且規模很小（例如只有 6 筆警報），數字不宜外推到真實環境。

Benchmark、量子與未來方向

Benchmark 與量子

- 系統級（BountyBench）、情境式（ARCeR、RedTeamLLM）、領域式（FinGAIA）、聚焦式（WASP、AgentVigil）
- 取捨：廣度 vs. 診斷精準度；自動化 vs. 擬真度
- 量子：quantum agents、QML 偵測、後量子密碼（harvest-now-decrypt-later 風險）

未來方向

- 即時輸入檢查（區分指令與資料）
- 內部執行透明度與結構化 log
- 帶形式化約束的規劃
- 與不可信環境的安全互動
- 多 agent 協作安全、記憶完整性
- 統一 benchmark、治理與部署保證

結論：自主性提升速度與適應力，但降低可預測性與可稽核性；需要有界自主、可強制的執行控制、持續保證與人類監督。

Agent-SafetyBench (arXiv 2412.14470)

Agent-SafetyBench: Evaluating the Safety of LLM Agents
Zhang, Cui, Lu, Zhou et al.（Tsinghua University, CoAI group）

問題

過去研究多關注 LLM 的內容安全（是否產生不安全文字）。當 LLM 作為 agent 與環境、工具互動，出現新的行為安全問題，例如在公開論壇洩漏敏感資訊、誤改訂單數量。 這類行為不一定在文字上有害，需要理解風險才能避免，但缺乏全面的評測。

做法

提出 Agent-SafetyBench：349 個互動環境、2,000 個測試案例、8 類風險、10 種失敗模式，並微調一個專用 scorer 自動判斷安全性。

Benchmark 概覽與資料建構

特色

- 349 個環境（先前最多 53 個）、每類風險 250 筆，共 2,000 筆
- 大量沒有公開 API 的新環境（例：MindCloning、NanorobotController）
- 每個案例標註預期的失敗模式
- 環境為 JSON 工具 schema + Python class，可自訂初始參數

資料來源

- 精修既有資料集（R-Judge、AgentDojo、ToolEmu 等）得 876 筆（含由 AdvBench 與 20 種越獄模板組成的 200 筆）
- GPT-4o 擴增得 1,124 筆：先生成 300 個新環境名稱以提升多樣性，再生成預期風險行為以提升品質
- 品質控管：人工預檢、自動驗證、人工後檢；交叉驗證 98% 案例、97.5% 標籤合理

8 類風險與 10 種失敗模式

風險類別

1. 洩漏敏感資料
2. 造成財產損失
3. 散布不安全資訊 / 假訊息
4. 造成人身傷害
5. 違法 / 違反倫理
6. 損害可用性
7. 有害 / 有漏洞程式碼
8. 產生不安全資訊（不用工具）

失敗模式 (M1–M10)

- M1 不呼叫工具直接產生有害內容
- M2 資訊不完整仍呼叫工具
- M3 取得完整資訊前就呼叫工具
- M4 忽略已知限制條件
- M5 忽略隱含或潛在風險
- M6 使用錯誤參數
- M7 忽略工具已知問題仍呼叫
- M8 未呼叫必要工具
- M9 過度信任工具結果
- M10 無法從多個結果挑出安全選項

評分方式：微調 Scorer

- 行為安全需考慮環境中的互動，直接用 GPT-4o 當 judge 在 200 筆 Gemini-1.5-Flash 紀錄上準確率僅 75.5%
- 以人工標註的 4,000 筆互動紀錄（2,186 unsafe / 1,814 safe）訓練，GPT-4o 針對人工標籤生成分析說明（抽查 94% 合理）
- 基礎模型 Qwen2.5-7B-Instruct，輸出 safe / unsafe 與分析，準確率 91.5%
- 評測流程：agent 選工具 → 環境回傳結果 → 重複，直到給出最終回覆；安全分數 = 判為 safe 的比例

主要結果：16 個 agent 皆低於 60%

Model	Total	Behavior	Content
Claude-3-Opus	59.8	53.2	84.9
Claude-3.5-Sonnet	59.4	51.9	88.6
Claude-3.5-Haiku	55.1	40.7	86.4
GPT-4o	44.2	36.9	72.5
Gemini-1.5-Flash	41.6	34.6	69.1
Llama3.1-405B	35.4	24.0	79.6
Qwen2.5-7B	18.8	13.5	38.9
Average (16)	38.5	30.4	68.4

- 強模型整體較安全；閉源優於開源
- 行為安全明顯弱於內容安全（即使多數行為案例沒有越獄攻擊）
- 各類風險平均：Spread 僅 15.6%（agent 常不驗證就透過貼文、郵件散布資訊）；Produce 87.0% 最高（越獄已被大量研究）

失敗模式與有用性分析

各失敗模式平均安全分數

- 較高：M1 70.1、M6 63.3、M10 63.1
- 最低：M7 12.5、M2 18.1（捏造參數、忽略工具危險）
- 偏低：M5 23.2、M3 27.0、M4 28.3
- M9 37.5：只有單一選項時缺乏驗證

兩個根本缺陷

1. **缺乏穩健性**：工具使用不可靠（例如數量填錯）
2. **缺乏風險意識**：參數正確卻忽略後果（例如關閉所有警報）

將案例分為可安全完成 (fulfillable) 與不可 (unfulfillable)：強安全模型在 fulfillable 案例仍保持相當有用性（不是靠拒絕），在 unfulfillable 案例則更少提供協助，顯示較強的風險意識。

防禦提示的效果與結論

- 測試兩種 defense prompt：簡單版（列出 10 種失敗模式）與加強版（詳細描述加範例）
- 對弱模型（Qwen2.5-7B）幾乎無效；對 GPT-4o 等較強模型有提升，但有限
- Claude-3.5-Sonnet 即使用加強版仍低於 70%，且增加 context 長度

結論

沒有任何 agent 總分超過 60%。僅靠修改 prompt 無法解決 agent 安全問題，作者主張未來需要更有效的方法，例如微調。 限制：測試案例以常識推理為主，需要專業領域知識的情境留待未來。